CS_33600

# ProcessBuilder

The `ProcessBuilder` class can implement every kind of I/O redirection implemented by a shell. `ProcessBuilder` can have each of the child's three standard streams,

- inherit from the parent, or
- redirect to a file, or
- connect to a pipe.

In this document we will explain how to write Java code that mimics the operation of each shell redirection operator, `<`, `>`, `>>`, `2>`, `2>>`, `|`, etc. In addition, we will see how to do some forms of I/O redirection that cannot be done from a shell.

The `ProcessBuilder` class works closely with two other classes, the `Process` class and the `ProcessBuilder.Redirect` class (a static nested class inside the `ProcessBuilder` class).

A `ProcessBuilder` object represents a program that can be started.

```
    ProcessBuilder pb = new ProcessBuilder("programName");
```

The `ProcessBuilder` object lets us configure the program before it starts. We use the `ProcessBuilder` object to determine how the streams of the (not yet) running process will be connected. Those future stream connections are represented by `ProcessBuilder.Redirect` objects stored in the `ProcessBuilder` object.

When a `ProcessBuilder` object starts its program, the new operating system process is represented by a `Process` object.

```
    Process p = pb.start();
```

That `Process` object lets us interact with the live process, for example, by performing additional stream redirections or by waiting for the process to terminate.

`ProcessHandle` and `ProcessHandle.Info` are two other classes in Java's "Process API". A `ProcessHandle` object can represent any running operating system process, not just those processes started with `ProcessBuilder` (which are represented by both a `Process` object and a `ProcessHandle` object). We use a `ProcessHandle` object to interact with an operating system process and we use a `ProcessHandle.Info` object to get information about an operating system process.

- https://docs.oracle.com/en/java/javase/21/core/process-api1.html
- https://link.springer.com/content/pdf/10.1007/978-1-4842-3546-1_10
- https://link.springer.com/content/pdf/10.1007/978-1-4842-3330-6_5

Here are all the classes in Java's "Process API".

- Process
- ProcessBuilder
- ProcessBuilder.Redirect
- ProcessBuilder.Redirect.Type
- ProcessHandle
- ProcessHandle.Info
- Runtime

All the example code mentioned in this document is in the sub folders called "io_redirection" and "creating_a_pipe" from the following zip file.

- http://cs.pnw.edu/~rlkraft/cs33600/for-class/streams_and_processes.zip

# I/O redirection using ProcessBuilder

Although `ProcessBuilder` can configure I/O redirection for a child process, its behavior is a bit unusual.

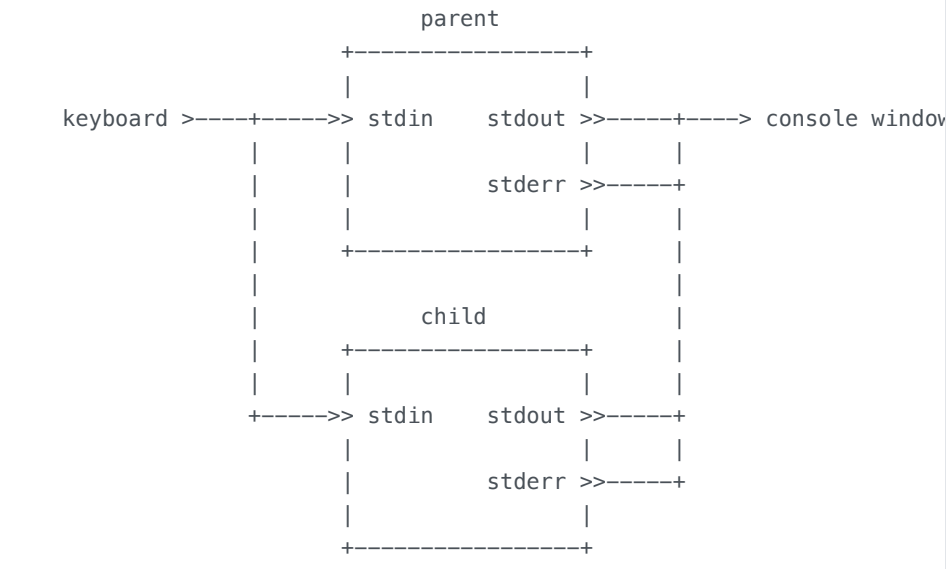
The most surprising aspect of `ProcessBuilder` is that when it creates a child process, the process's three standard streams seem not to be connected to anything. For example, create a file "Parent.java"

containing the following two class definitions. If you compile and run this program, the child's output will *not* appear in the console window.

```
class Child {
   public static void main(String[] args) {
      System.out.println("Where does this standard output end up?");
      System.err.println("Where does this standard error end up?");
   }
}

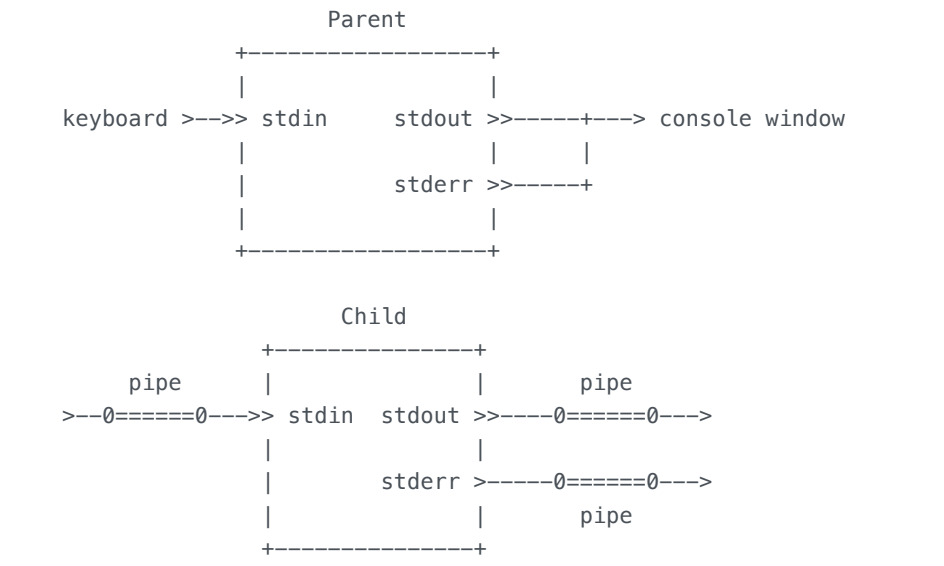

public class Parent {
   public static void main(String[] args) throws Exception {
      System.out.println("Starting java Child process ...");
      final Process p = new ProcessBuilder("java", "Child")
                            .start();
      p.waitFor();
      System.out.println("Child process terminated.");
   }
}
```

The default behavior in other programming languages is for the child process to inherit (and share) the parent process's three standard streams, as shown in the following picture. In particular, if the parent process is a console application, and the child is also a console application (not a GUI program), then the child can take over the console I/O in a normal way.




But that is not how Java's `ProcessBuilder` works. By default, when `ProcessBuilder` creates a process, the child's standard streams are each connected to a pipe, and the pipes are not

connected to anything. The following picture shows what the streams look like for a newly created process.



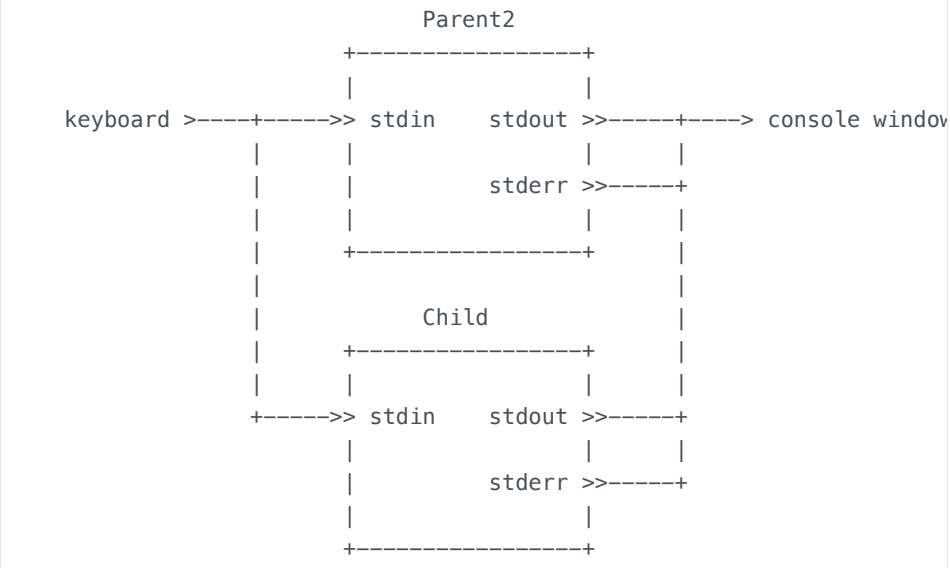

The answer to the question "Where does this standard output end up?" is that it ends up in the pipe buffer attached to the child's standard output stream. And the answer to the question "Where does this standard error end up?" is that it ends up in the pipe buffer attached to the child's error stream. For now, it would seem that this data is stuck in those buffers, but we will soon see how to get it out.

The simplest way to fix this is to use the `inheritIO()` method. This makes `ProcessBuilder` behave like most other systems.

```
public class Parent2 {
   public static void main(String[] args) throws Exception {
      System.out.println("Starting java Child process ...");
      final Process p = new ProcessBuilder("java", "Child")
                            .inheritIO()
                            .start();
      p.waitFor();
      System.out.println("Child process terminated.");
   }
}
```

- ProcessBuilder.inheritIO()

When we use `inheritIO()`, we get a picture that looks like this (without the default pipes attached to the child's standard streams).



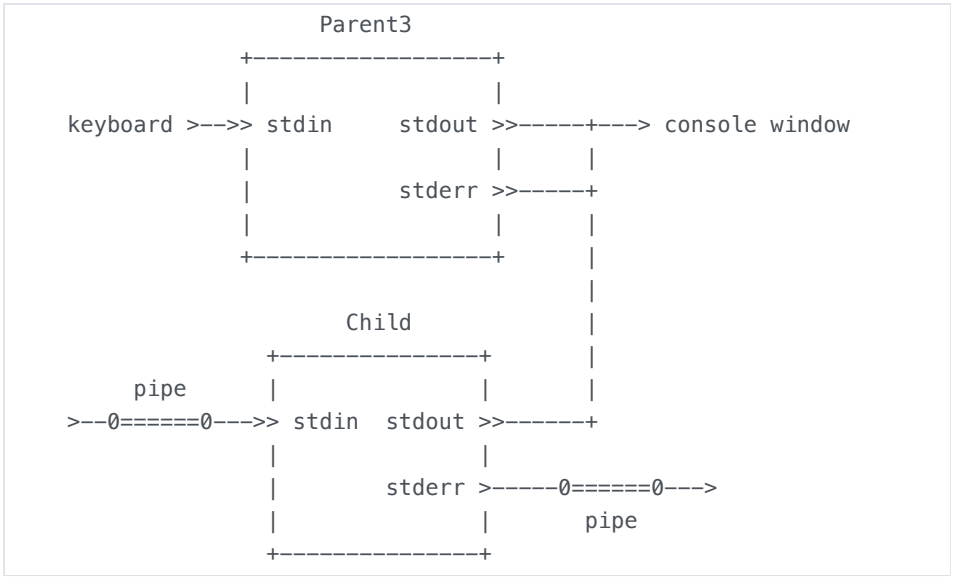

We don't have to connect all three streams in the same way. The `ProcessBuilder` API lets us choose the connection type for each standard stream independently.

For example, we can let the child process inherit the parent's standard output stream while leaving the child's other two streams connected to the default pipes.

To inherit the standard output stream, we use the `redirectOutput()` method with the `ProcessBuilder.Redirect.INHERIT` parameter.

```
public class Parent3 {
   public static void main(String[] args) throws Exception {
      System.out.println("Starting java Child process ...");
      final Process p = new ProcessBuilder("java", "Child")
                            .redirectOutput(ProcessBuilder.Redirect.INH
                            .start();
      p.waitFor();
      System.out.println("Child process terminated.");
   }
}
```

The above code creates the following picture of the child's standard streams. When we run the program, the child's standard output appears in the console window but the child's standard error remains stuck in the pipe buffer.




- ProcessBuilder.redirectOutput(ProcessBuilder.Redirect)
- ProcessBuilder.Redirect.INHERIT
- ProcessBuilder.Redirect.PIPE

We can use the `ProcessBuilder` methods `redirectInput(File)` and `redirectOutput(File)` to redirect the child's standard streams to files.

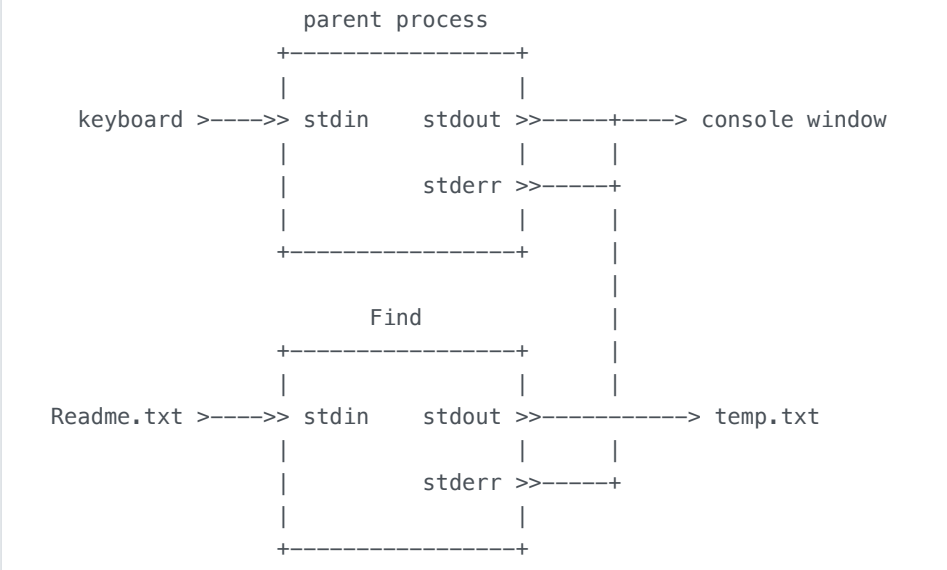
Consider the following command-line that uses the Java program `Find.java` from the "filter_programs" directory.

```
    > java Find process < Readme.txt > temp.txt
```

Here is how we implement this command-line using `ProcessBuilder`.

```
    Process p = new ProcessBuilder("java",
                                   "Find",
                                   "process")
                    .redirectInput(new File("Readme.txt"))   // < Readm
                    .redirectOutput(new File("temp.txt"))    // < temp.
                    .redirectError(ProcessBuilder.Redirect.INHERIT)
                    .start();
```

This creates the following picture.




We can test this in JShell. Open a command-line prompt in the "filter_programs" folder, compile the program "Find.java", start JShell, and then paste the following code into the `jshelll` prompt.

Because of the way JShell works, a child process of JShell cannot share any of JShell's standard input, output, or error streams. So the example code redirects the child's error streams to a file.

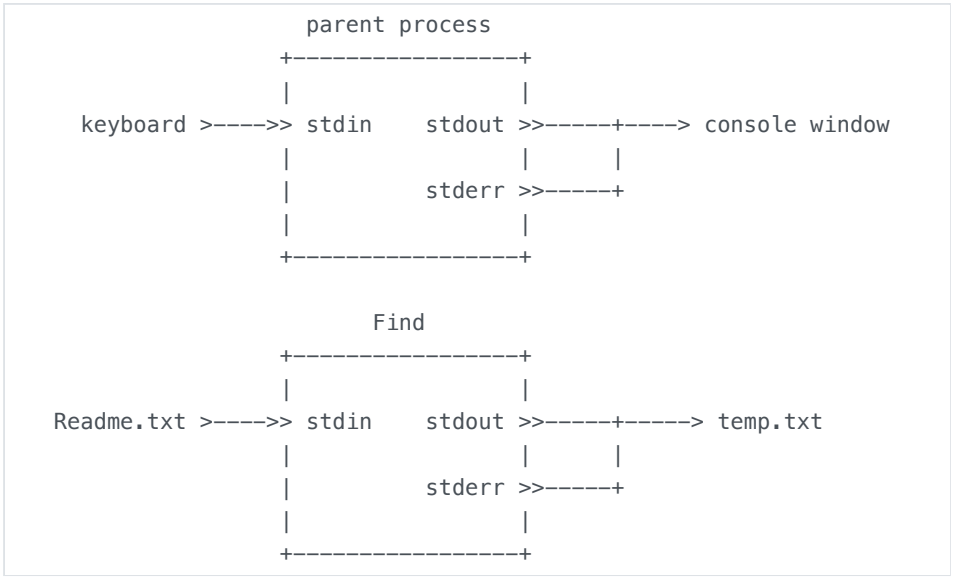
```
var p = new ProcessBuilder("java", "Find", "process").
            redirectInput(new File("Readme.txt")).
            redirectOutput(new File("temp.txt")).
            redirectError(new File("errors.txt")).
            start()
```

This should create a new file, "temp.txt", in the "filter_programs" folder containing the results from searching the file "Readme.txt" for the string "process".

Try misspelling the name "Find" as "Fid". What happens? Why?

Try misspelling the name "Readme.txt" as "Redme.txt". What happens? Why?

We can combine the child's error stream with its standard output stream by using the `redirectErrorStream(boolean)` method.




Try the following code in `jshell` and try causing different kinds of errors. Where do the error messages end up?

```
var p = new ProcessBuilder("java", "Find", "process").
            redirectInput(new File("Readme.txt")).
            redirectOutput(new File("temp.txt")).
            redirectErrorStream(true).
            start()
```

The shell has the `>>` append redirection operator that writes new data at the end of the current data in a file. We can implement this kind of redirection using the `ProcessBuilder.Redirect appendTo(File)` method. The `appenTo()` method is a static method in the class `Redirect` which is a static nested class in the class `ProcessBuilder`.

If you run the following code in JShell after running the above code, then the results of searching "Readme.txt" for the string "filter" should be in "temp.txt" after the results of searching for the string "process".

```
var p2 = new ProcessBuilder("java", "Find", "filter").
             redirectInput(new File("Readme.txt")).
             redirectOutput(ProcessBuilder.Redirect.appendTo(new File(
             redirectErrorStream(true).
             start()
```

**Exercise:** What is the advantage of this first block of code over the second block of code?

```
var p1 = new ProcessBuilder("java", "Find", "process").
             redirectInput(new File("Readme.txt")).
             redirectOutput(new File("temp1.txt")).
             redirectErrorStream(true).
             start()
```

```
var p2 = new ProcessBuilder("java", "Find", "process").
             redirectInput(new File("Readme.txt")).
             redirectOutput(new File("temp2.txt")).
             redirectError(new File("temp2.txt")).
             start()
```

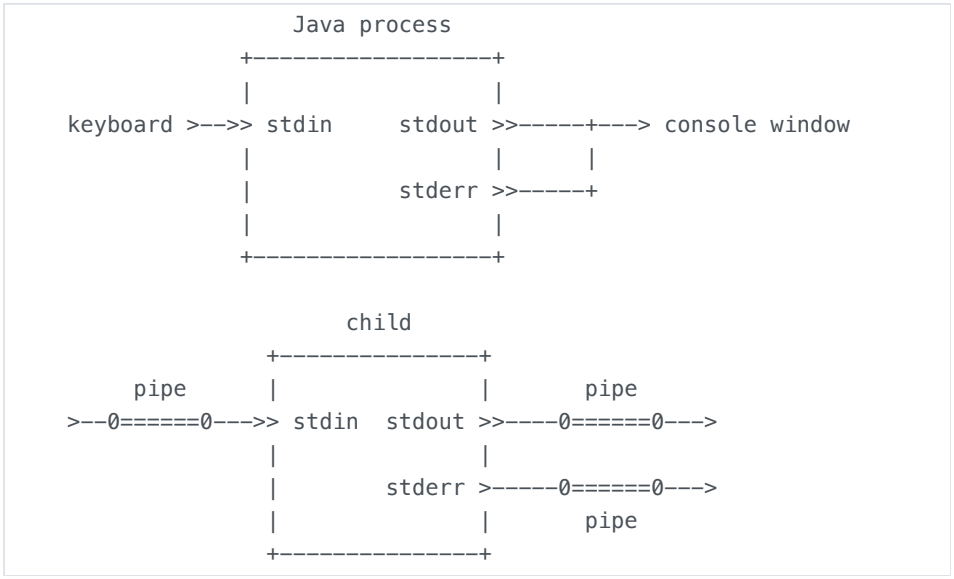
Read the details of the `redirectInput(File)`, `redirectOutput(File)`, `redirectError(File)`, `Redirect.appendTo(File)`, and `redirectErrorStream(boolean)` methods in the `ProcessBuilder` Javadocs.

- ProcessBuilder.redirectInput(File)
- ProcessBuilder.redirectOutput(File)
- ProcessBuilder.redirectError(File)
- ProcessBuilder.redirectErrorStream(boolean)
- ProcessBuilder.Redirect.appendTo(File)

Here is a brief summary of how `ProcessBuilder` redirects correlate to shell syntax.

```
var pb = new ProcessBuilder("myProgram")
             .redirectInput(new File("input.txt"))
             .redirectOutput(new File("output.txt"))
             .redirectOutput(Redirect.appendTo(new File(output.txt")))
             .redirectError(new java.io.File("errors.txt"))
             .redirectErrorStream(true)
```

We mentioned earlier that when `ProcessBuilder` creates a process, its default configuration looks like this.




That's because this code,

```
    Process p = new ProcessBuilder("child")
                    .start();
```

is equivalent to the following.

```
    Process p = new ProcessBuilder("child")
                    .redirectInput(ProcessBuilder.Redirect.PIPE)
                    .redirectOutput(ProcessBuilder.Redirect.PIPE)
                    .redirectError(ProcessBuilder.Redirect.PIPE)
                    .start();
```

The `ProcessBuilder` constructor makes those three method calls.

We mentioned that the most common alternative to the above configuration is this code,

```
    Process p = new ProcessBuilder("child")
                    .inheritIO()
                    .start();
```

which is equivalent to this.

```
    Process p = new ProcessBuilder("child")
                    .redirectInput(ProcessBuilder.Redirect.INHERIT)
                    .redirectOutput(ProcessBuilder.Redirect.INHERIT)
                    .redirectError(ProcessBuilder.Redirect.INHERIT)
                    .start();
```

The `inheritIO()` method is just a convenience method for those three method calls.

The `ProcessBuilder.Redirect` class is the abstraction that represents each kind of I/O redirection that a parent process can choose for a child process. The parent process makes its selections by calling `redirectInput()`, `redirectOutput()`, and `redirectError()` with a `ProcessBuilder.Redirect` object as the parameter.

The parameter to `redirectOutput()` or `redirectError()` can be one of,

- the constant object `ProcessBuilder.Redirect.PIPE`,
- the constant object `ProcessBuilder.Redirect.INHERIT`,
- a return value from `ProcessBuilder.Redirect.toFile(File)`,
- a return value from `ProcessBuilder.Redirect.appendTo(File)`.

The parameter to `redirectInput()` can be one of,

- the constant object `ProcessBuilder.Redirect.PIPE`,
- the constant object `ProcessBuilder.Redirect.INHERIT`,
- a return value from `ProcessBuilder.Redirect.fromFile(File)`.

A method call of the form

```
    redirectInput( ProcessBuilder.Redirect.fromFile(File) )
```

can be replaced by this convenience method.

```
    redirectInput( File )
```

Similarly, a method call of the form

```
    redirectOutput( ProcessBuilder.Redirect.toFile(File) )
```

can be replaced by this convenience method.

```
    redirectOutput( File )
```

In the next section we will explain the `Redirect.PIPE` option.

# Create a pipeline using ProcessBuilder

In this section we will show how a Java process can create a pipeline of two other processes (the other two processes need *not* be Java processes). We will show how to build several different kinds of pipelines. First, we will show how a Java process can mimic the way a shell process creates a pipeline. Second, we will show how a Java process can start a child process and feed data into the child and draw data from the child. Third, we will show how a Java process can start a pipeline of two child processes and feed data into the beginning of the pipeline and draw data from the end of the pipeline.

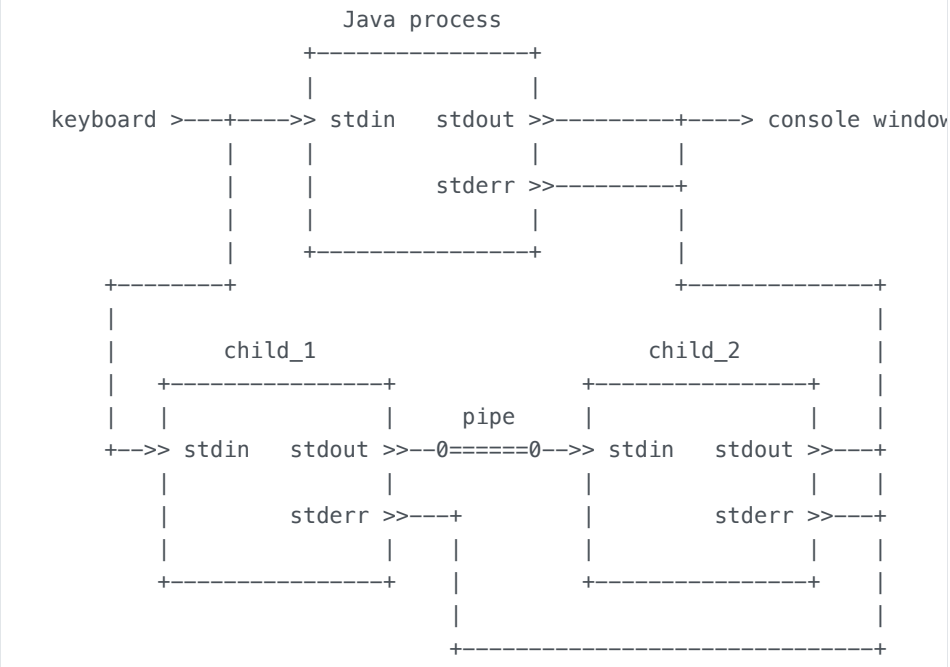
The example programs mentioned in this section are all in the sub folder called "creating_a_pipe" from the following zip file.

- http://cs.pnw.edu/~rlkraft/cs33600/for-class/streams_and_processes.zip

We want to mimic how a shell process creates a pipe between two child processes. That is, we want our process to mimic this shell command.

```
> child_1 | child_2
```

Our process must create a pipe and two child processes, then share our process's standard input stream with the first child, share our process's standard output and error streams with the second child, connect the standard output stream of the first child to the input of the pipe, and finally connect the output of the pipe to the standard input stream of the second child.



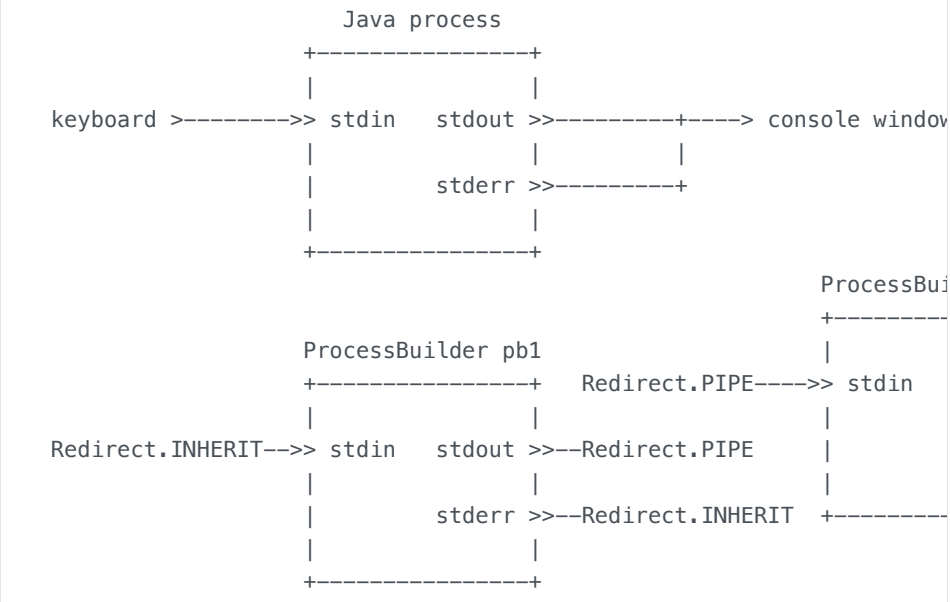

We will build up the code for this picture in steps so that we can see what each step does.

First, we create a `ProcessBuilder` object for each child processes.

```
final ProcessBuilder pb1 = new ProcessBuilder("child_1")
                               .redirectInput(ProcessBuilder.Redirec
                               .redirectError(ProcessBuilder.Redirec

final ProcessBuilder pb2 = new ProcessBuilder("child_2")
                               .redirectOutput(ProcessBuilder.Redire
                               .redirectError(ProcessBuilder.Redirec
```

Notice that this code creates `ProcessBuilder` objects, not `Process` objects, so these objects do not represent running processes. We can illustrate what this code does with the following picture. The ProcessBuilder objects in this picture are not operating system objects and they are not Java objects either. This picture is meant to help us understand what `ProcessBuilder` does. Think of these ProcessBuilders as "proto-objects" for the operating system processes (or "proto-processes").




The first child process will share its standard input and error streams with the parent process. The second child process will share its standard output and error streams with the parent process. The ProcessBuilder objects shown above represent these future stream connections using `ProcessBuilder.Redirect.INHERIT` objects. These `` `ProcessBuilder.Redirect `` objects act as "place holders", in the `ProcessBuilder` object, for the actual stream connections that will be created when the child processes are started.

Notice that there are no actual pipe objects yet, just `Redirect.PIPE` placeholder objects. And notice that we did not mention the `Redirect.PIPE` objects in our code because they are the default type of connection.

Now we add the code that actually creates and starts the two child processes. This code also creates the needed operating system pipe and creates all the stream connections specified above.

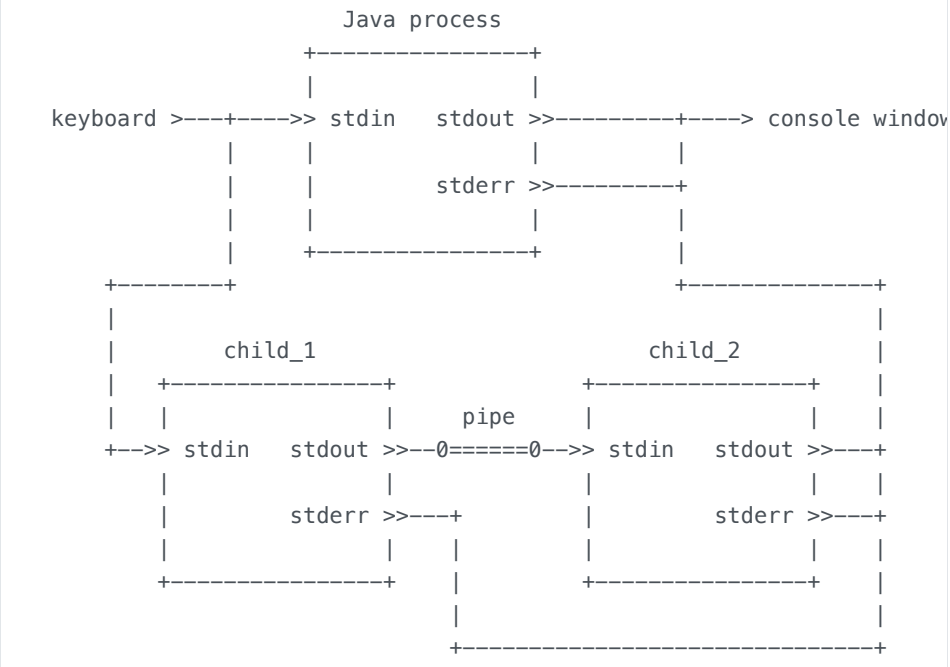
This is all done with a call to the `startPipeline()` method, giving it a `List` of all the `ProcessBuilder` objects that represent the pipeline processes.

```
final ProcessBuilder pb1 = new ProcessBuilder("child_1")
                               .redirectInput(ProcessBuilder.Redirec
                               .redirectError(ProcessBuilder.Redirec

final ProcessBuilder pb2 = new ProcessBuilder("child_2")
                               .redirectOutput(ProcessBuilder.Redire
                               .redirectError(ProcessBuilder.Redirec

final List<ProcessBuilder> builders = java.util.Arrays.asList(pb1, p
final List<Process> pipeline = ProcessBuilder.startPipeline(builders
```

This code creates the picture that we want.




Here is a complete Java program that creates a pipeline. This program needs to be run from within the "filter_programs" folder.

```
import java.util.List;
import java.util.Arrays;
import java.io.IOException;

public class TestPipe
{
   public static void main(String[] args) throws IOException, Interrupt
   {
      final ProcessBuilder pb1 = new ProcessBuilder("java", "ToUpperCas
                                     .redirectInput(ProcessBuilder.Redi
                                     .redirectError(ProcessBuilder.Redi

      final ProcessBuilder pb2 = new ProcessBuilder("java", "Reverse")
                                     .redirectOutput(ProcessBuilder.Red
                                     .redirectError(ProcessBuilder.Redi

      final List<ProcessBuilder> builders = Arrays.asList(pb1, pb2);
      final List<Process> pipeline = ProcessBuilder.startPipeline(build

      for (final Process p : pipeline)
      {
         p.waitFor();
      }
   }
}
```

When you run this program, it will wait for you to type some text into its standard input stream (using the keyboard). It will print the result from your first line of input text in the console window, and then wait for you to type another line of input text (this program does not "prompt" you for input, it just waits for you to type). When you do not want to type any more input, use Ctr-z in Windows, or Ctr-d in Linux, to close the program's input stream (don't use Ctrl-c, that kills the program).

**Exercise:** What happens if you remove the for-loop at the end of the "TestPipe.java" program and then run the program in a shell? (Its important that you run the modified program from a shell.)

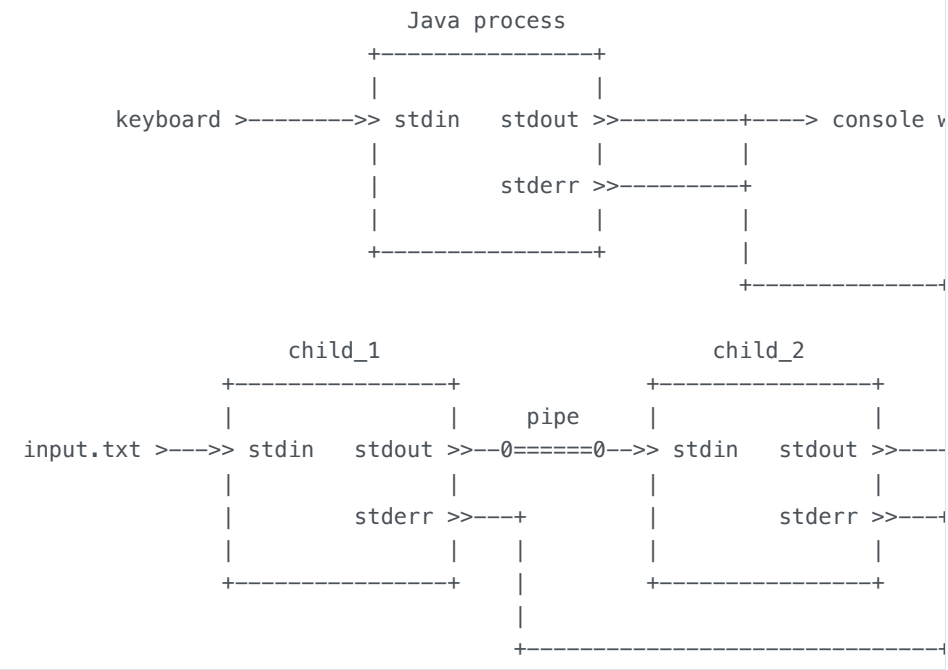
```
    filter_programs> java TestPipe
```

Hint: Draw a picture of all the streams involved in this command-line, including the shell process (cmd or bash). What happens when the `TestPipe` process terminates? (Hint: "TestPipe" terminating does not terminate the pipeline.)

A variation on the basic pipeline command is to redirect the pipeline's input and output streams to files.

```
    > child_1 < input.txt | child_2 > output.txt
```

This command-line has the following picture.




This requires a simple modification to the above code. In the `redirectInput()` and `redirectOutput()` methods we replace `Redirect.INHERIT` with `File` objects.

The code below implements the following pipeline command in JShell.

```
    > java ToUpperCase < Readme.txt | java Reverse > temp.txt
```

The code assumes that JShell is being run from the "filter_programs" folder.

Because of the way JShell works, a child process of JShell cannot share any of JShell's standard input, output, or error streams. So the example code redirects the children's error streams to a file.

```
var pb1 = new ProcessBuilder("java", "ToUpperCase").
               redirectInput(new File("Readme.txt")).
               redirectError(ProcessBuilder.Redirect.appendTo(new File(
var pb2 = new ProcessBuilder("java", "Reverse").
               redirectOutput(new File("temp.txt")).
               redirectError(ProcessBuilder.Redirect.appendTo(new File(
var builders = java.util.Arrays.asList(pb1, pb2);
var pipeline = ProcessBuilder.startPipeline(builders);
for (final Process p : pipeline)
{
   p.waitFor();
}
```

**Exercise:** Use `ProcessBuilder` to create a three stage pipeline.

**Exercise:** Use `ProcessBuilder` to implement the bash `|&` operator.

```
    > child_1 |& child_2
```

This operator pipes the first child's standard error stream through the pipeline along with the first child's standard output stream. Draw a picture of what the streams should look like for this command.

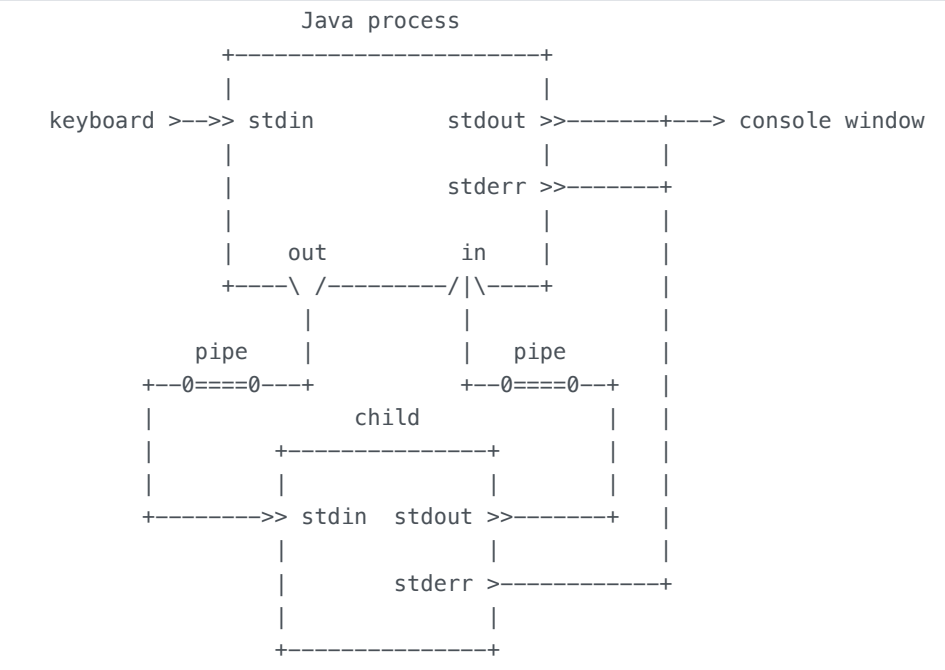
**Exercise:** The following bash command-line is supposed to pipe the standard error stream of `child_1` into the standard input stream of `child_2` while the standard output stream of `child_1` remains shared with the shell's standard output stream. Try to implement this configuration with `ProcessBuilder`.

```
    $ child_1 3>&1 1>&2 2>&3 | child_2
```

# Create a worker process using ProcessBuilder

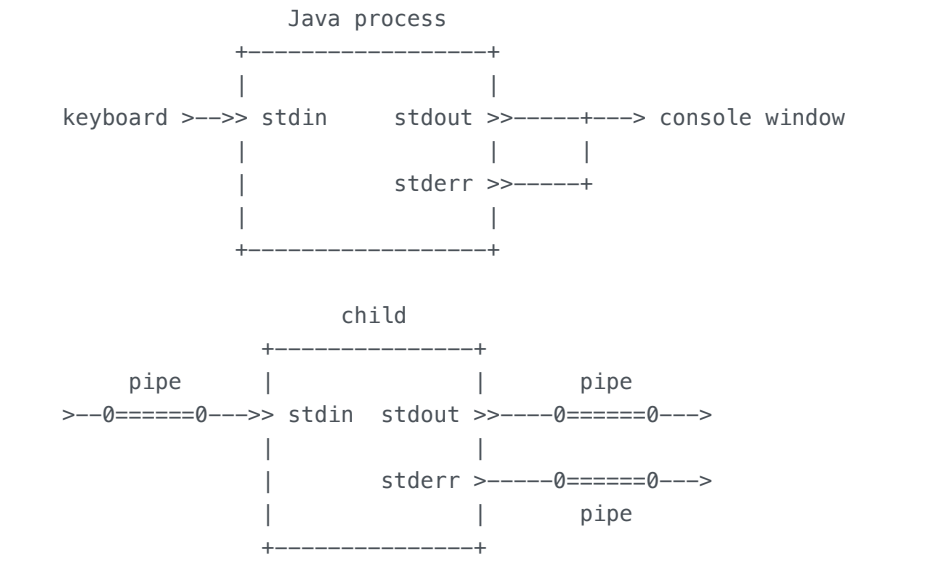
It is interesting to know how to create the above kind of pipeline because that is what a shell does. But those pipelines are not very useful for the parent process. The parent process cannot use those pipelines to have the child pipeline do work for the parent process. The parent process does not have an IPC link between it and the child processes. In many situations we want the child pipeline to do work for the parent. So the parent needs some kind IPC with the pipeline.

For our second step, we want a Java process to create the following picture. This is not a pipeline, but it shows how a parent process can start a child process, send it some data to work on, and then read the transformed data.




We will once again build up the code for this picture in small steps so that we can see what each step does.

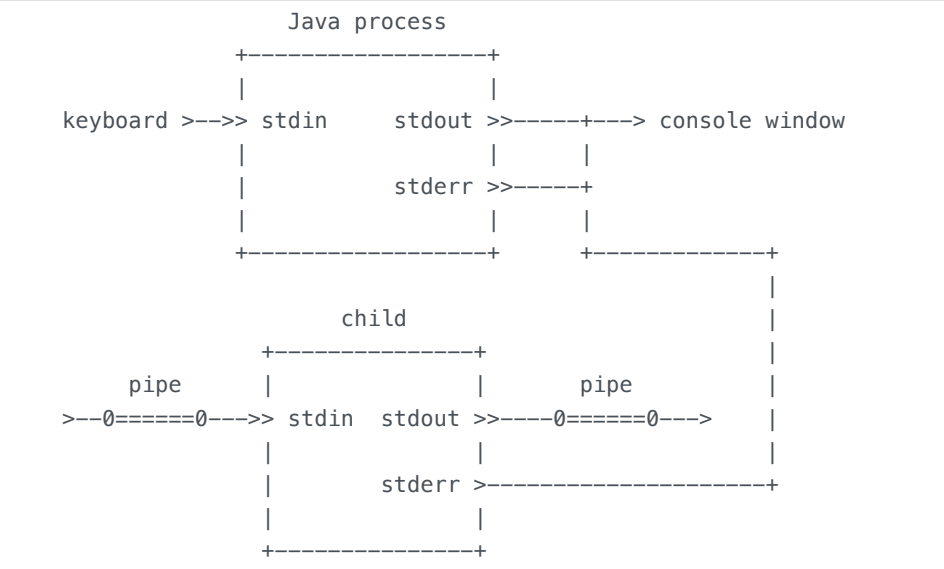
Recall that when `ProcessBuilder` creates a child process, by default it is connected to three pipes.




We can see that we want to use the two pipes connected to the child's standard input and output streams, but we do not want the pipe connected to the error stream. This code creates the following

picture.

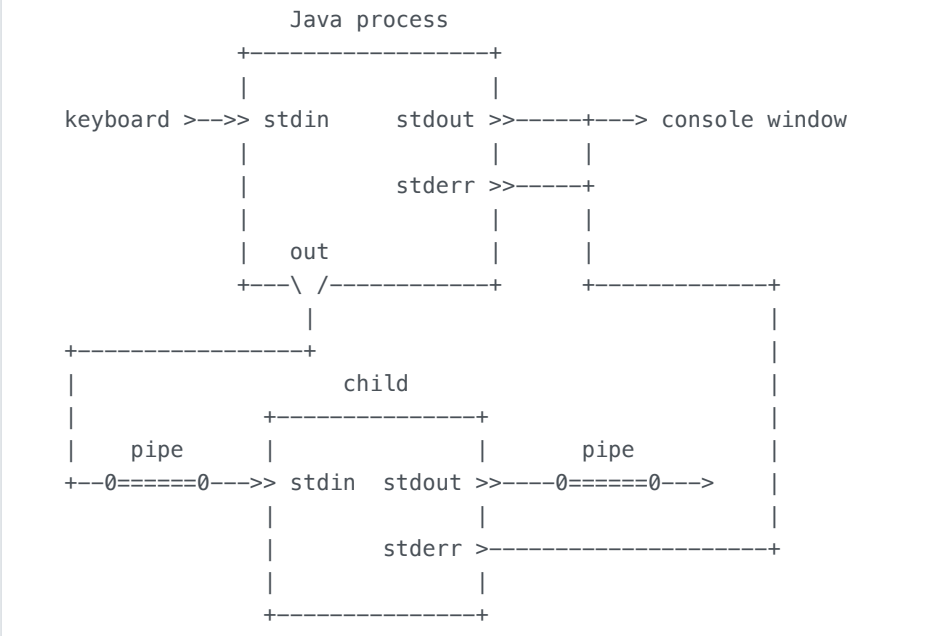
```
final Process p = new ProcessBuilder("child")
                      .redirectError(ProcessBuilder.Redirect.INHERIT
                      .start();
```




The `Process` method `getOutputStream()` creates a new output stream in the parent process that is connected to the input of the pipe connected to the child's standard input stream. This is kind of confusing. First of all, you need to call this method on the `Process` object. That means this redirection happens *after* the child process starts running. Second, the name of the method is `getOutputStream()` but it "gets" a connection to the child's input stream.

The following code creates the following picture.

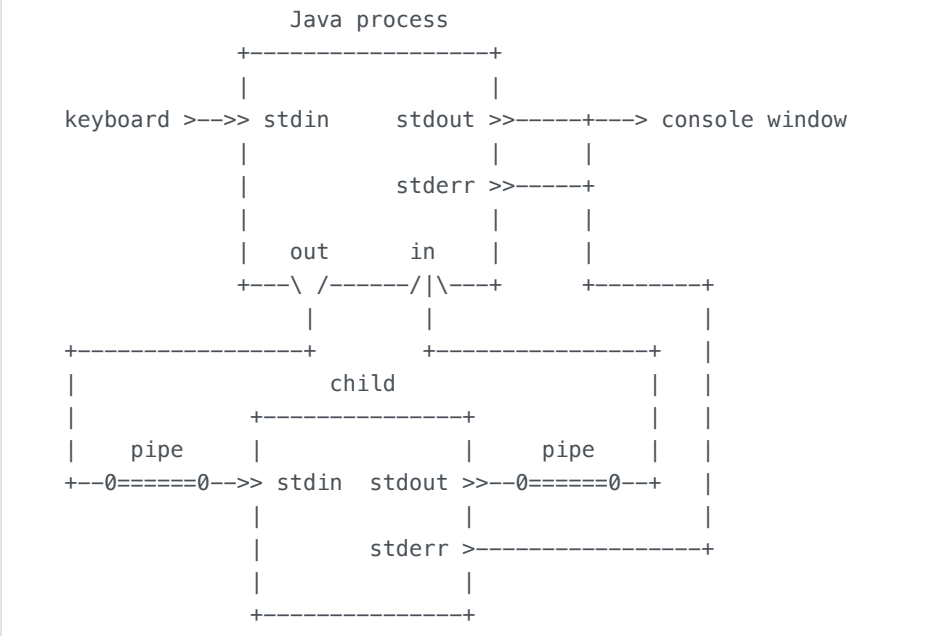
```
final Process p = new ProcessBuilder("child")
                      .redirectError(ProcessBuilder.Redirect.INHERIT
                      .start();
final OutputStream out = p.getOutputStream();
```




We are almost where we want to be. The `Process` method `getInputStream()` creates a new input stream in the parent process that is connected to the output of the pipe connected to the child's standard output stream. You need to call this method on the `Process` object, so this redirection happens *after* the child process starts running. The name of the method is `getInputStream()` but it "gets" a connection to the child's output stream.

The following code creates the following picture, which is what we want.

```
final Process p = new ProcessBuilder("child")
                      .redirectError(ProcessBuilder.Redirect.INHERIT
                      .start();
final OutputStream out = p.getOutputStream();
final InputStream  in  = p.getInputStream();
```




- Process.getInputStream()
- Process.getOutputStream()

We can test this code in JShell. Go to the folder that contains all the filter programs, "filter_programs", compile all the filter programs, and then start a command-prompt window in that folder and start a JShell session. Then copy and past this block of code into the `jshell` prompt.

```
var p = new ProcessBuilder("java", "ToUpperCase").
                    redirectError(ProcessBuilder.Redirect.INHERIT).
                    start();
var out = p.getOutputStream();
var in  = p.getInputStream();
out.write( "hello".getBytes() )
out.close()
in.read()
in.read()
in.read()
in.read()
in.read()
```

The numbers that the `read()` method returns are the integer values of the ASCII upper case characters `H`, `E`, `L`, `L`, and `O`.

Here is a slightly better way to read the data out of the child process.

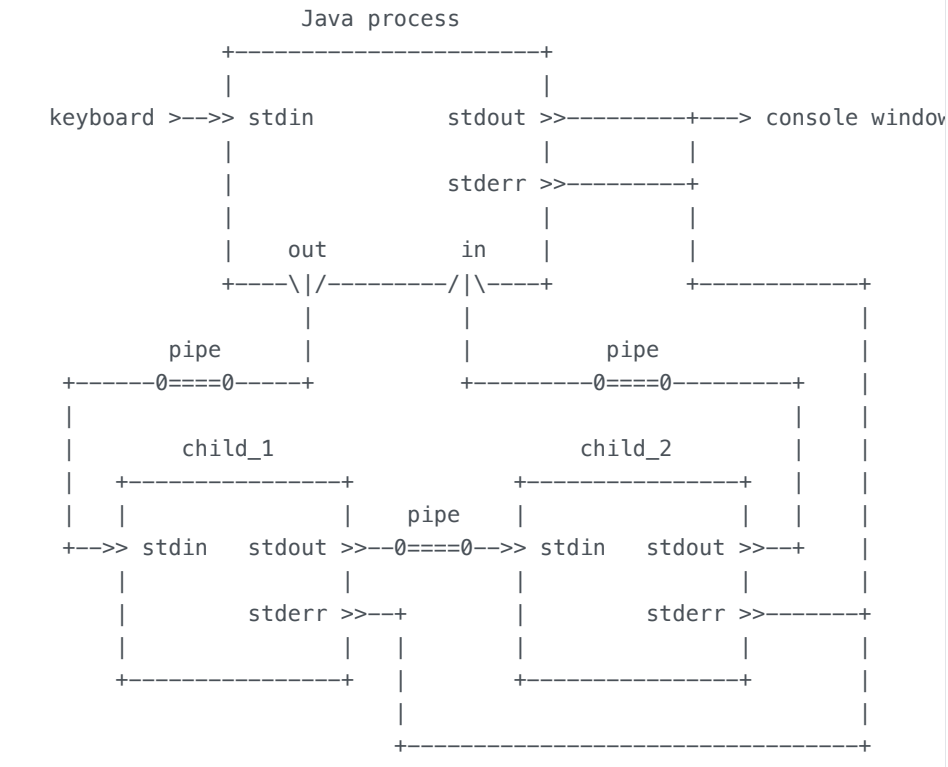
```
var p = new ProcessBuilder("java", "ToUpperCase").
                     redirectError(ProcessBuilder.Redirect.INHERIT).
                     start();
var out = p.getOutputStream();
var in  = p.getInputStream();
out.write( "hello".getBytes() )
out.close()
var bytes = new byte[5]
in.read(bytes)
bytes
new String(bytes)
```

Notice how we have to convert our input `String` into a `byte` array, then convert the output `byte` array back into a `String`. The streams are strictly streams of bytes.

Try changing the input data to the child process. Make sure that you get the proper output data. Try changing which filter program is used as the child process. Try writing a `String` or a `double` directly into the child's input stream. What happens?

# Create a worker pipeline using ProcessBuilder

For the third step, we want a Java process to create the following pipeline.



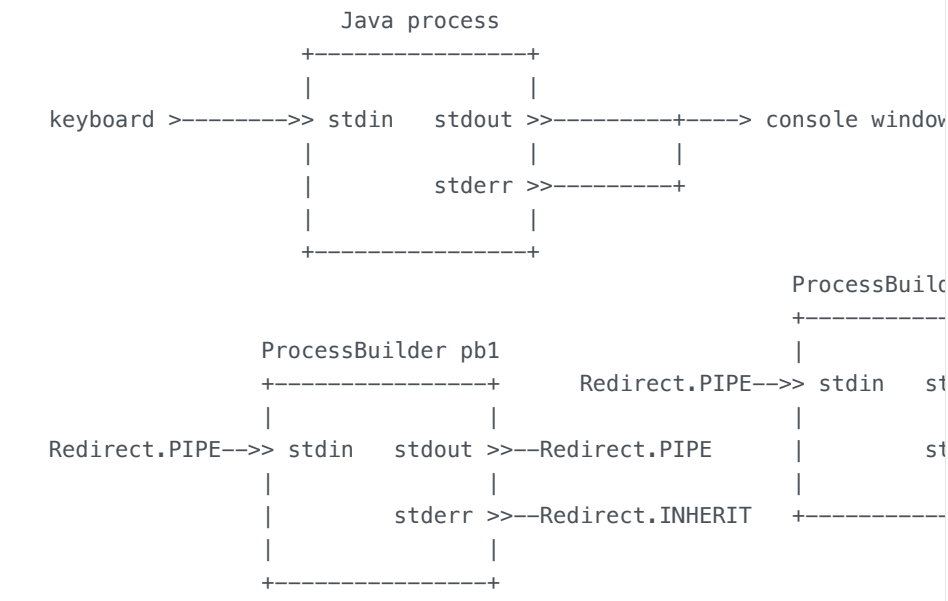
The Java process should create a new output stream, a new input stream, two child processes, and a pipe, and then redirect the first child's standard input to the new output stream, redirect the second child's standard output to the new input stream, and connect the two child processes with the pipe.

The code for this pipeline is a combination of the code from the previous two steps. First, we create two `ProcessBuilder` objects.

```
   final ProcessBuilder pb1 = new ProcessBuilder("child_1")
                                .redirectError(ProcessBuilder.Redirec

   final ProcessBuilder pb2 = new ProcessBuilder("child_2")
                                .redirectError(ProcessBuilder.Redirec
```

The `ProcessBuilder` objects are in this configuration, with `ProcessBuilder.Redirect` objects in place of streams. Notice that there are more `Redirect.PIPE` objects this time.




Then we start the pipeline out of those two `ProcessBuilder` objects.

```
   final ProcessBuilder pb1 = new ProcessBuilder("child_1")
                                .redirectError(ProcessBuilder.Redirec

   final ProcessBuilder pb2 = new ProcessBuilder("child_2")
                                .redirectError(ProcessBuilder.Redirec

   final List<ProcessBuilder> builders = java.util.Arrays.asList(pb1, p
   final List<Process> pipeline = ProcessBuilder.startPipeline(builders
```

This starts the child processes and causes two of the `ProcessBuilder.Redirect` objects to be replaced by streams and four are replaced by pipe objects. Notice that two of the pipe objects are not yet fully connected.

```
                      Java process
                    +----------------+
                    |                |
   keyboard >-------->> stdin   stdout >>---------+----> console windov
                    |                |          |
                    |         stderr >>---------+
                    |                |          |
                    +----------------+          |
                                                +--------------------

                         child_1                      child_2
                  +----------------+           +----------------+
         pipe     |                |    pipe   |                |    pi
     >--0====0-->> stdin   stdout >>--0====0-->> stdin   stdout >>--0==
                  |                |           |                |
                  |         stderr >>----+     |         stderr >>-----
                  |                |     }     |                |
                  +----------------+     |     +----------------+
                                         |
                                         +------------------------------
`

Finally, we connect the parent process to the input and output streams
from the pipeline.

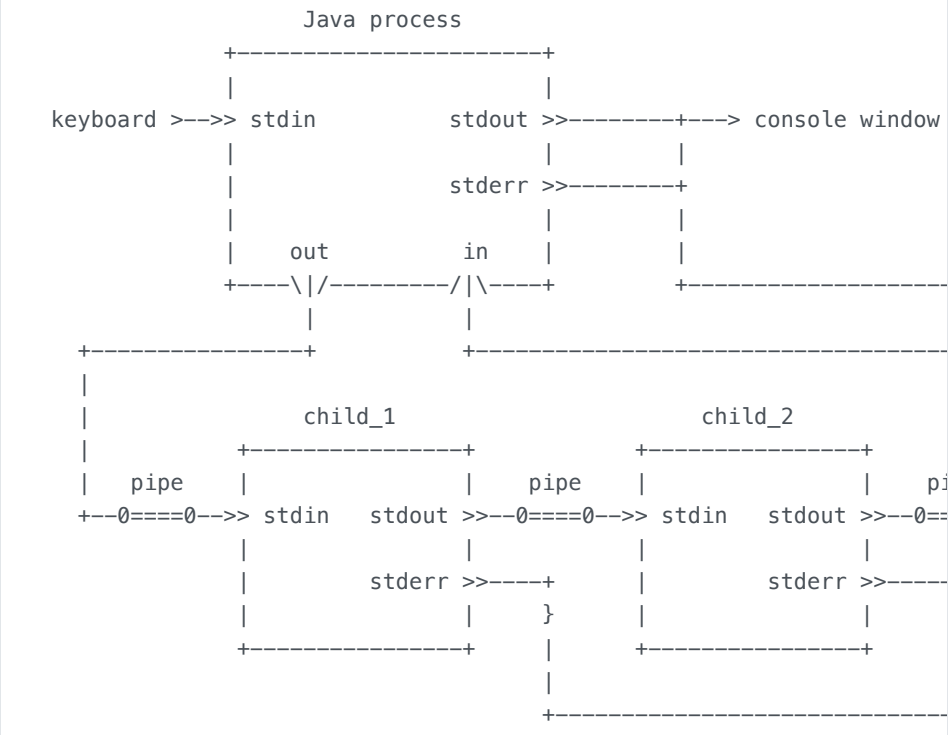
```java
   final ProcessBuilder pb1 = new ProcessBuilder("child_1")
                                .redirectError(ProcessBuilder.Redirec

   final ProcessBuilder pb2 = new ProcessBuilder("child_2")
                                .redirectError(ProcessBuilder.Redirec

   final List<ProcessBuilder> builders = java.util.Arrays.asList(pb1, p
   final List<Process> pipeline = ProcessBuilder.startPipeline(builders

   final OutputStream out = pipeline.get(0).getOutputStream();
   final InputStream  in  = pipeline.get(1).getInputStream();
```

That creates the configuration that we want.
```




Here is a block of code that implements this picture using JShell.

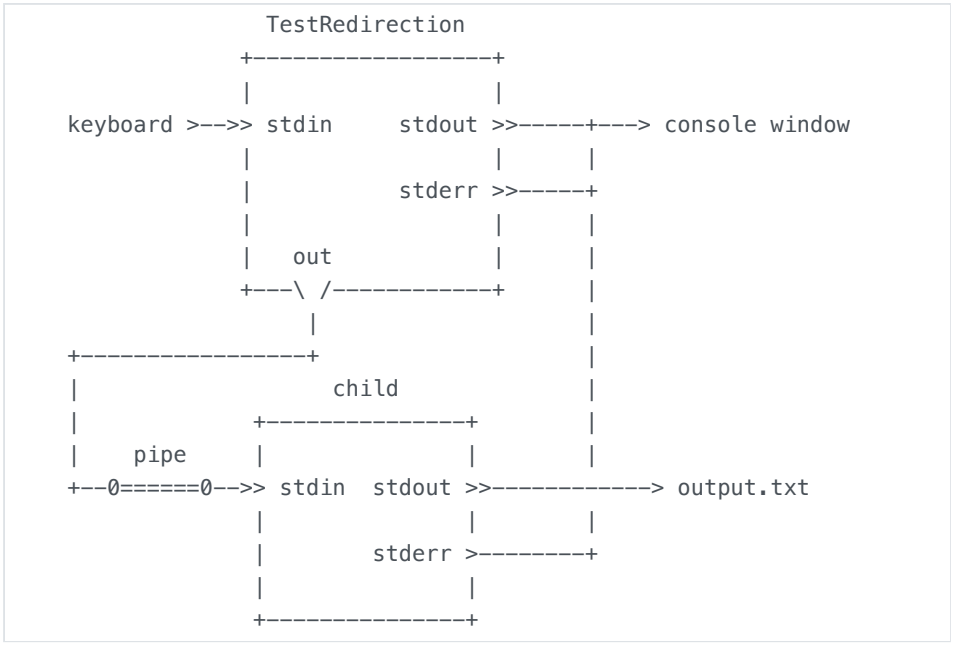
```
var pb1 = new ProcessBuilder("java", "ToUpperCase").
              redirectError(ProcessBuilder.Redirect.INHERIT);

var pb2 = new ProcessBuilder("java", "DoubleN", "3").
              redirectError(ProcessBuilder.Redirect.INHERIT);

List<ProcessBuilder> builders = java.util.Arrays.asList(pb1, pb2);
List<Process> pipeline = ProcessBuilder.startPipeline(builders);

var out = pipeline.get(0).getOutputStream();
var in  = pipeline.get(1).getInputStream();
out.write( "hello".getBytes() )
out.close()
var bytes = new byte[15]
in.read(bytes)
bytes
new String(bytes)
```

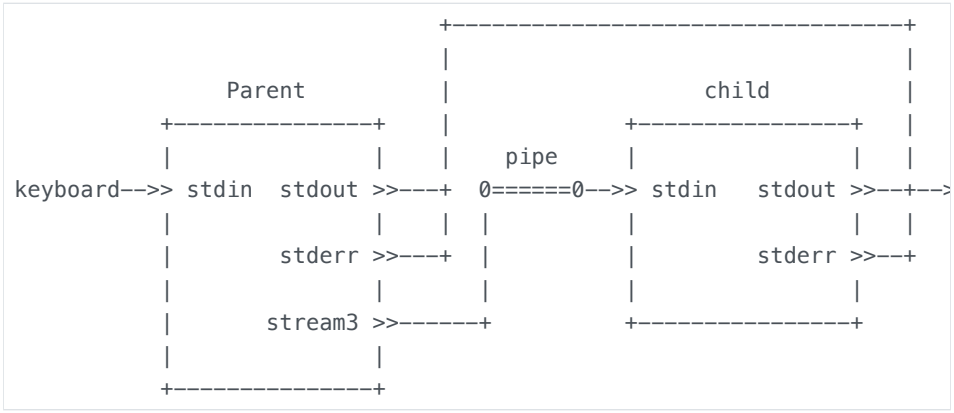
**Exercise:** Write a Java program called `TestRedirection.java` that creates the following picture. The child process should be another Java program. The name of the child's class should be a command-line argument to the `TestRedirection.java` program. The `TestRedirection.java` program should read data from its standard input stream and copy the data to its stream called `out`.




Explain what the following command-line will do.

```
    > java TestRedirection Double < Reverse.java
```

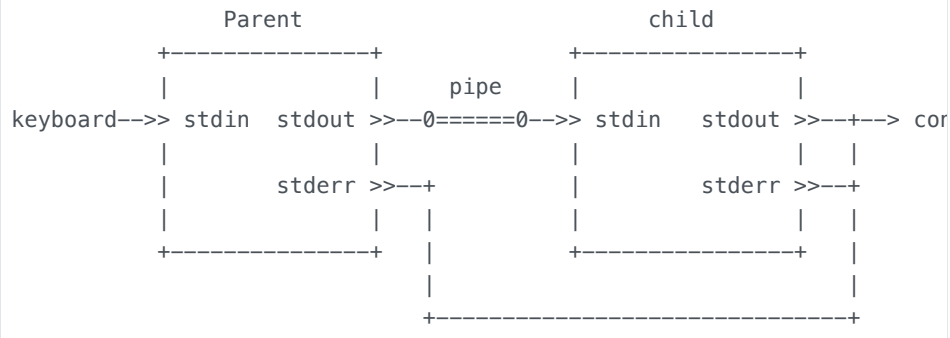
**Exercise:** Create the following pipeline of a parent with its child.




This is the kind of pipeline that a shell process uses when we pipe a "builtin" command into a program. For example, in cmd,

```
    > dir | find "hello"
```

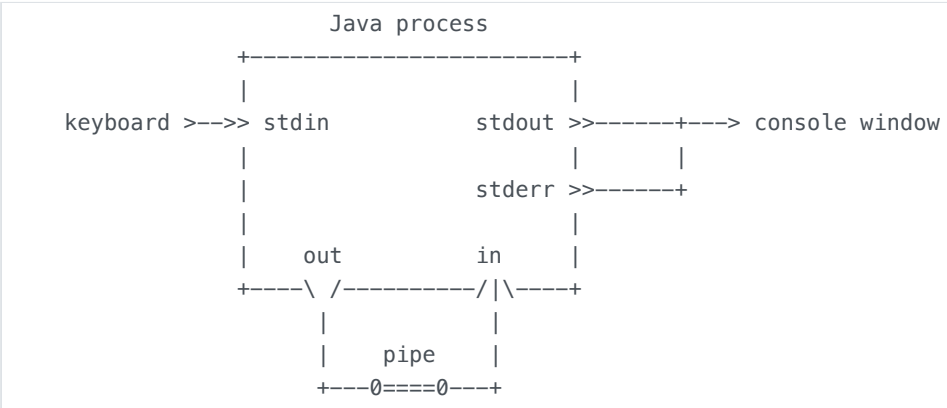
**Exercise:** Create the following pipeline of a parent with its child. (Hint: Use `System.setOut()`).




How does this compare to the pipeline in the previous problem? Does either pipeline structure have an advantage over the other? (Hint: What happens when the child terminates?)

# PipedInputStream and PipedOutputStream

One interesting aspect of how Java works with pipes is that we cannot use Java to create the following configuration in which there is no child process, just a pipe object that is connected to a new output stream and a new input stream.




We can create this configuration using the C language. It may not seem to be useful. In the C language, this is an intermediate step to creating a pipe between two child processes. But this configuration is useful for testing and demonstrating ideas about streams, and it can also be used as a communication channel between two threads within the Java process.

In fact, this configuration is useful enough that Java has a way to create something similar to it. In the `java.io` package there are two specialized stream classes called `PipedInputStream` and `PipedOutputStream`.

- java.io.PipedInputStream

- java.io.PipedOutputStream

These are low level streams that can be connected to each other to mimic the structure of a pipe. The following two lines of code create a "pipe" between `pos` and `pis`. Whatever bytes are written into `pos` can be read from `pis`.

```
    PipedOutputStream pos = new PipedOutputStream();
    PipedInputStream  pis = new PipedInputStream(pos); // connect pis t
```

The two lines of code create the following steams. The "buffer" is not a pipe. The "buffer" is part of the `PipedInputStream` object.

```
                      Java process
                +-----------------------+
                |                       |
    keyboard >-->> stdin          stdout >>------+---> console window
                |                       |      |
                |                 stderr >>------+
                |                       |
                |    pos          pis   |
                +----\ /----------/|\----+
                      |           |
     PipedOutputStream |           | PipedInputStream
                      |           |
                      +---0====0---+
                          buffer
```

We can experiment with this configuration in JShell. Open a command-prompt window and start a JShell session. Copy-and-paste the following block of code into the `jshell` prompt.

```
var pos = new PipedOutputStream();
var pis = new PipedInputStream(pos); // connect pis to pos
pos.write( "hello".getBytes() )
pis.read()
pis.read()
pis.read()
pis.read()
pis.read()
```

The numbers that the `read()` method returns are the integer values of the ASCII characters `h`, `e`, `l`, `l`, and `o`.

If we try to read one more byte from the `PipedInputStream`, the `read()` method will block on the empty buffer and wait for something to write a byte into the `PipedOutputStream`. But, since there is no other prompt that we can use to call `pos.write()`, another call to `pis(read()` will get blocked

(stuck) forever. We would need to use Ctrl-C to kill the JShell process. This is a concern with using this kind of "pipe". If we are not careful, it can lead to "deadlock". But this kind of "pipe" can be useful for demonstrating, in JShell, how Java streams work.

We can read the data out of `pis` in a slightly better way.

```
var pos = new PipedOutputStream();
var pis = new PipedInputStream(pos);
pos.write( "hello".getBytes() )
var bytes = new byte[5]
pis.read(bytes)
bytes
new String(bytes)
```

If you make the `bytes` array larger, then `read()` will get stuck (try it).

- https://web.mit.edu/java_v1.0.2/www/tutorial/java/io/pipedstreams.html
- https://en.wikipedia.org/wiki/Deadlock_(computer_science)

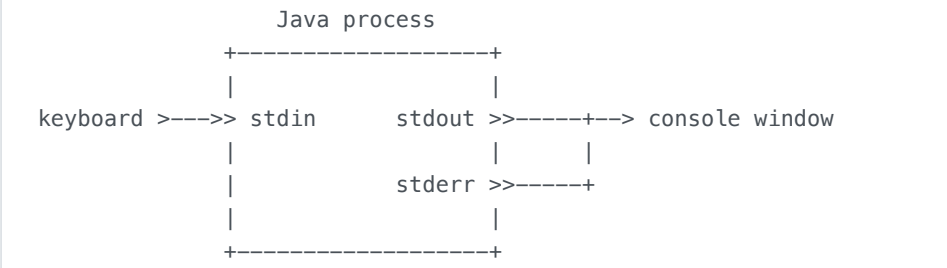
## System.setIn() and System.setOut()

The example programs mentioned in this section are all in the sub folder called "io_redirection" from the following zip file.

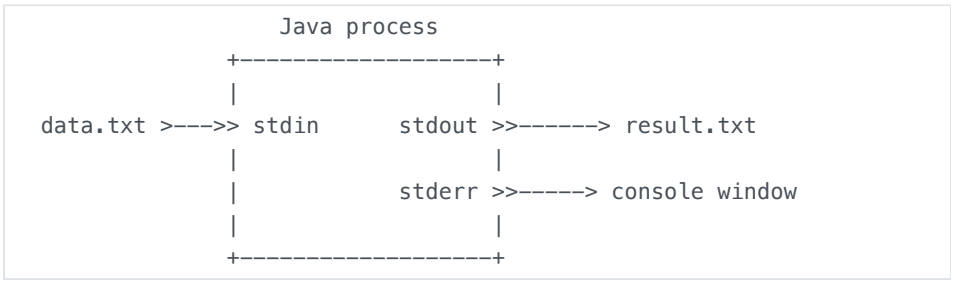
- http://cs.pnw.edu/~rlkraft/cs33600/for-class/streams_and_processes.zip

So far, everything in this document has been about a parent process redirecting the standard streams of a *child* process. If a process wishes to, it can redirect its *own* standard streams.

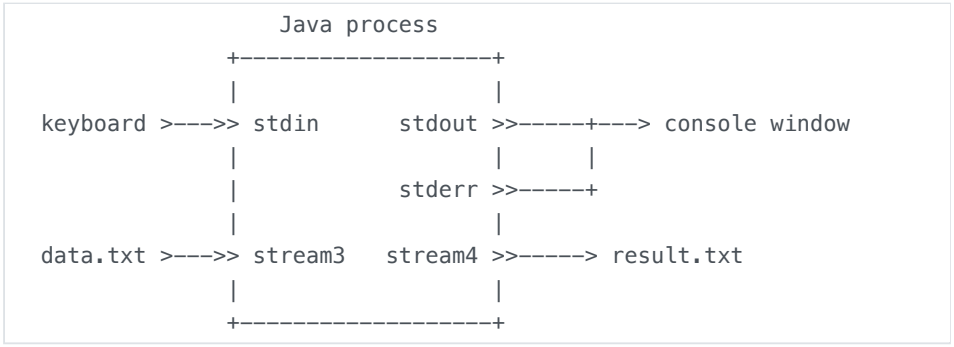
A process might be running with the following configuration of its standard streams.



We want to see how this process can, while it is running, change its standard streams to look like the following picture, with stdin and stdout connected to specific files chosen by the process (the names of the files could be coded directly into the process or they could be command-line arguments to the process).




This does not seem like a very useful thing to do. It is almost exactly the same idea as opening new streams for reading and writing (as in the picture below). The main difference between the above picture and the picture below is that the process above can use `System.in` and `System.out` to read and write the files "data.txt" and "result.txt". The process does not need to create any new streams. The process below must explicitly open an `InputStream` to "data.txt" for `stream3` and an `OutputStream` to "result.txt" for `stream4`.




This kind of redirection is useful in some testing situations. It can also be used to redirect `System.out` to a buffered output stream in order to speed up the `System.out.print()` method (by as much as 10 times). A process may want to redirect its own stdin and stdout streams so that it can have a child process inherit those redirected streams.

This kind of redirection is not done using the `ProcessBuilder` class. We do it using three static methods in the `java.lang.System` class.

- java.lang.System.setIn()
- java.lang.System.setOut()
- java.lang.System.setErr()
- java.lang.System

Here is a complete Java program that shows how we can use `setIn()` and `setOut()`.

```
import java.io.InputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.PrintStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;

public class TestSetInSetOut
{
   public static void main(String[] args) throws FileNotFoundException,
   {
      System.out.println("Copying \"Readme.txt\" to \"ReadmeCopy.txt\".

      // Save the original streams so that they can be restored.
      final InputStream inSaved  = System.in;
      final PrintStream outSaved = System.out;

      // Set System.in to be the file "Readme.txt".
      System.setIn( new FileInputStream( "Readme.txt" ) );

      // Set System.out to be the file "ReadmeCopy.txt.
      System.setOut( new PrintStream( new FileOutputStream( "ReadmeCopy

      // Create a Scanner object to make it easier to use System.in
      final Scanner scanner = new Scanner(System.in);

      // Echo every line of input from stdin to stdout.
      while ( scanner.hasNextLine() )
      {
         final String oneLine = scanner.nextLine();
         System.out.println( oneLine );
      }

      scanner.close();
      System.in.close(); // This may be redundant.
      System.out.close();

      // Restore the original values of System.in and System.out.
      System.setIn(inSaved);
      System.setOut(outSaved);

      System.out.println("Done copying \"Readme.txt\" to \"ReadmeCopy.t
      System.out.print("Press Enter to continue ...");
      System.in.read();  // Show that keyboard input has been restored.
      System.out.println("Bye.");
   }
}
```

Notice how the code restores the original versions of `System.in` and `Systerm.out`. This makes it perfectly safe to modify the standard streams, use the modified streams, and then restore the original streams.

Here is a version of the above code that will work in JShell as long as JShell is started in a folder that includes a "Readme.txt" file.

```
System.out.println("Copying \"Readme.txt\" to \"ReadmeCopy.txt\".")
var inSaved  = System.in
var outSaved = System.out
System.setIn( new FileInputStream( "Readme.txt" ) )
System.setOut( new PrintStream( new FileOutputStream( "ReadmeCopy.txt"
var scanner = new Scanner(System.in)
while ( scanner.hasNextLine() ) {
   final String oneLine = scanner.nextLine();
   System.out.println( oneLine );
}
scanner.close()
System.in.close()
System.out.close()
System.setIn(inSaved)
System.setOut(outSaved)
System.out.println("Done copying \"Readme.txt\" to \"ReadmeCopy.txt\".'
```

**Exercise:** What does the following command-line do?

```
    > java TestSetInSetOut > temp.txt
```

What does the following command-line do?

```
    > java TestSetInSetOut < TestSetInSetOut.java > temp.txt
```

Explain in detail exactly what this command-line causes to happen. Be sure to notice that the shell does I/O redirection on its child process and then the child process does I/O redirection on itself.

**Exercise:** In what ways are the following two command-lines similar and in what ways do they differ?

```
    > java TestSetInSetOut
    > java Echo < Readme.txt > RadmeCopy.txt
```

Here is example of what can be done using this kind of redirection combined with redirections handled by `ProcessBuilder`. The following program uses `ProcessBuilder` to filter its `System.out` stream through three different filters.

```
public class TestSetOut
{
   public static void main(String[] args) throws IOException,
                                                  InterruptedException
   {
      System.out.println("This should print normally.");

      final PrintStream outSaved = System.out;

      final Process p1 = new ProcessBuilder("java", "-cp", "filters.jar
                             .redirectInput(ProcessBuilder.Redirect.PI
                             .redirectOutput(ProcessBuilder.Redirect.I
                             .redirectError(ProcessBuilder.Redirect.IN
                             .start();
      System.setOut(new PrintStream(p1.getOutputStream()));
      System.out.println("This should print in all upper case.");
      System.out.close(); // Try commenting this out. Or try replacing
      p1.waitFor();       // Try commenting this out.

      final Process p2 = new ProcessBuilder("java", "-cp", "filters.jar
                             .redirectInput(ProcessBuilder.Redirect.PI
                             .redirectOutput(ProcessBuilder.Redirect.I
                             .redirectError(ProcessBuilder.Redirect.IN
                             .start();
      System.setOut(new PrintStream(p2.getOutputStream()));
      System.out.println("This should print in reverse.");
      System.out.close(); // Try commenting this out.
      p2.waitFor();       // Try commenting this out.

      final Process p3 = new ProcessBuilder("java", "-cp", "filters.jar
                             .redirectInput(ProcessBuilder.Redirect.PI
                             .redirectOutput(ProcessBuilder.Redirect.I
                             .redirectError(ProcessBuilder.Redirect.IN
                             .start();
      System.setOut(new PrintStream(p3.getOutputStream()));
      System.out.println("This should print one word-per-line.");
      System.out.close(); // Try commenting this out.
      p3.waitFor();       // Try commenting this out.

      System.setOut(outSaved);
      System.out.println("This should print normally again.");
   }
}
```

This program can also be used to demonstrate a few other ideas. If you comment out any of the `System.out.close()` methods, then a child will wait forever for more input on its standard input stream. Calling `System.out.close()` makes the operating system send the child process the end-of-file condition on its input stream, so the child knows that it's time to terminate.

You can also try replacing the `System.out.close()` method with `System.out.flush()`. The program will still hang forever, but it will act a bit differently (why?).

If you comment out all the `waitFor()` methods, then all four process can run simultaneously. But they all share the same output stream. In that case their output gets all jumbled up in the output stream.

**Exercise:** Draw a reasonable picture of what the parent process, its child processes, and all of their streams look like when the above program runs.

**Exercise:** Modify the above program so that it creates all three child processes and then alternates using them. The parent sends a line to one child, then a line to another child, etc. When the parent has no more lines to send to the children, then it closes their streams. This can be made to work, but there is a problem. What is the problem?

Here is an interesting example of why we might want to redirect `System.out`. The default version of `System.out` is an unbuffered output stream. This makes it easier to use, but also makes it very slow. Below is a complete program that compares the speed of the unbuffered `System.out` with a buffered version. This program uses `System.setOut()` to replace an unbuffered `PrintStream` with a buffered one.

```
public class TestSystemOutSpeed
{
   public static void main(String[] args)
   {
      // Use the default version of System.out.
      final long startTime1 = System.currentTimeMillis();
      for (int i = 0; i < 50_000; ++i)
      {
         System.out.println("This is a timed test of the speed of the S
      }
      final long stopTime1 = System.currentTimeMillis();

      // Create a buffered version of System.out.
      System.setOut(new PrintStream(
                           new BufferedOutputStream(System.out,
                                                    4096),
                           false));
      long startTime2 = System.currentTimeMillis();
      for (int i = 0; i < 50_000; ++i)
      {
         System.out.println("This is a timed test of the speed of the b
      }
      final long stopTime2 = System.currentTimeMillis();

      System.out.printf("Wall-clock time: %,d milliseconds\n", stopTime
      System.out.printf("Wall-clock time: %,d milliseconds\n", stopTime
      System.out.flush(); // Buffered streams must be flushed.
   }
}
```

When you run this program you should see that the buffered version of `System.out` is about 10 times faster than the original, unbuffered version. This is a very large difference in speed. It shows that the convenience of using an unbuffered output stream comes at a very high performance price.

Below is a JShell version of this code. The timing values in JShell are quite a bit different than in a plain console window. The default JShell version of `System.out` is faster than the default console version of `System.out` (JShell is probably using some kind of buffering). But even in JShell, the buffered `PrintStream` is faster than the unbuffered one, but only twice as fast, and not as fast as the buffered `PrintStream` running in a plain console window. On my computer, the buffered `PrintStream` running in a plain console window was twice as fast as the buffered `PrintStream` running in JShell.

```
long startTime1 = System.currentTimeMillis();
for (int i = 0; i < 50_000; ++i) {
   System.out.println("This is a timed test of the speed of the System.
}
long stopTime1 = System.currentTimeMillis();

System.setOut(new PrintStream(new BufferedOutputStream(System.out, 4096
long startTime2 = System.currentTimeMillis();
for (int i = 0; i < 50_000; ++i) {
   System.out.println("This is a timed test of the speed of the buffere
}
long stopTime2 = System.currentTimeMillis();

System.out.printf("Wall-clock time: %,d milliseconds\n", stopTime1 - st
System.out.printf("Wall-clock time: %,d milliseconds\n", stopTime2 - st
```

In a later document we will say a lot more about buffers and explain why an unbuffered output stream is easier to use but a buffered output stream is faster.

- https://www.baeldung.com/java-testing-system-out-println
- https://medium.com/@AlexanderObregon/how-javas-system-out-println-actually-works-79556a9c2837
- https://luckytoilet.wordpress.com/2010/05/21/how-system-out-println-really-works/

I/O redirections done using `System.setIn()` and `System.setOut` do not always work the way we might expect them to. In particular, these I/O redirections do not seem to be inheritable. That is, if a parent redirects its own `System.in` and then uses `ProcessBuilder` to create a child that inherits the parent's standard input stream, then the child will inherit the input stream the parent had when the parent was created, not the parent's current, redirected, `System.in` stream.

Below is the code for an example program. If you run this program with this command-line

```
    > java TestSetIn
```

then the parent's original `System.in` is the keyboard. When this program runs, the child process is expecting its input from the keyboard. The child does not get its input from the file "TestSetIn_data.txt", which is where the parent redirected its `System.in`.

If you run this program with this command-line

```
    > java TestSetIn < Readme.txt
```

then the parent's original `System.in` is the file "Readme.txt". And when this program runs, the child process gets its input from "Readme.txt". The child does *not* get its input from the file "TestSetIn_data.txt".

```
class Child {
   public static void main(String[] args) throws IOException {
      final Scanner scanner = new Scanner(System.in);
      for (int i = 0; i < 3; ++i) {
         final String oneLine = scanner.nextLine();
         System.out.println("Child : " + oneLine);
      }
      scanner.close();
      System.out.println("Child process done.");
   }
}

public class TestSetIn {
   public static void main(String[] args) throws FileNotFoundException,
                                                  IOException,
                                                  InterruptedException {
      System.setIn(new FileInputStream("TestSetIn_data.txt"));
      System.out.println("Starting Child process ...");
      final Process p = new ProcessBuilder("java", "Child")
                            .inheritIO()
                            .start();
      p.waitFor();

      final Scanner scanner = new Scanner(System.in);
      for (int i = 0; i < 3; ++i) {
         final String oneLine = scanner.nextLine();
         System.out.println("Parent: " + oneLine);
      }
      scanner.close();
      System.out.println("Parent process done.");
   }
}
```

**Exercise:** Write a program that tests the inheritance of `System.setOut`. If a process uses `System.setOut` to redirect its own `System.out` stream and then uses `ProcessBuilder` to create a child that inherits `System.out`, what stream does the child inherit?

I do not know if this is a bug or a feature. If `System.setIn` is designed this way to prevent some kind of problem, then it could be a "feature". Otherwise, I would consider it to be a "bug".

- https://en.wikipedia.org/wiki/Bug_(engineering)#%22It's_not_a_bug,_it's_a_feature
- https://en.wikipedia.org/wiki/Undocumented_feature
- https://www.wired.com/story/its-not-a-bug-its-a-feature/

---